

Projekt: Software Engineering

Projektdokumentation

MietManager

Portfolio von Michael Bangas

Matrikelnummer: IU14132885

Kursbezeichnung: Projekt: Software Engineering (DLMCSPSE01_D)

Studiengang: M.Sc. Informatik

IU Internationale Hochschule

25. September 2026

Inhaltsverzeichnis

1 Projektübersicht	1
1.1 Problemstellung und Motivation	1
1.2 Zielgruppe und Kundennutzen	1
1.3 Funktionsumfang	2
1.4 Angestrebtes Ergebnis	2
2 Theoretische Grundlagen	2
2.1 Risikomanagement in Softwareprojekten	2
2.2 Anforderungspriorisierung nach MoSCoW	3
2.3 Iteratives und inkrementelles Vorgehen	3
3 Risikomanagement	3
4 MoSCoW-Priorisierung	4
5 Zeitplanung	5
5.1 Woche 1 – Konzeptionsphase, Teil 1 (27.07.–02.08.2026)	6
5.2 Woche 2 – Konzeptionsphase, Teil 2 (03.08.–09.08.2026)	6
5.3 Woche 3 – Erarbeitungs-/Reflexionsphase (10.08.–16.08.2026)	6
5.4 Woche 4 – Finalisierungsphase (17.08.–23.08.2026)	7
6 GitHub-Repository	7
7 Benutzeranleitung	7
7.1 Installation und Start	7
7.2 Bedienung	8
8 Fazit	9
Literaturverzeichnis	11

Abbildungsverzeichnis

5.1 Projektzeitplan 6

Tabellenverzeichnis

3.1	Risikomanagement	4
4.1	MoSCoW-Priorisierung des Funktionsumfangs	5

1 Projektübersicht

Der folgende Abschnitt beschreibt zunächst das zugrunde liegende Managementproblem privater Kleinvermieter, bevor darauf aufbauend die im Rahmen dieses Projekts entwickelte Anwendung MietManager mit Zielgruppe und Kundennutzen, geplantem Funktionsumfang sowie angestrebtem Ergebnis vorgestellt wird.

1.1 Problemstellung und Motivation

Private Kleinvermieter mit wenigen vermieteten Wohneinheiten stehen jährlich vor der Aufgabe, aus den anfallenden Betriebskosten eine korrekte Nebenkostenabrechnung je Mietvertrag zu erstellen. Bei manueller Bearbeitung mit Tabellenkalkulation oder auf Papier ist dieser Prozess fehleranfällig und zeitaufwendig. Kostenpositionen müssen nach dem vereinbarten Umlageschlüssel, etwa Fläche, Personenzahl oder Anzahl Einheiten, korrekt auf die einzelnen Mietverträge verteilt, den bereits geleisteten Vorauszahlungen gegenübergestellt und abschließend in einem versandfertigen Schreiben an die Mieter kommuniziert werden. Anders als größere Hausverwaltungen verfügen private Kleinvermieter dafür in der Regel über keine spezialisierte Software. Kommerzielle Hausverwaltungslösungen sind meist auf einen deutlich größeren Immobilienbestand ausgelegt. Diese Lücke zwischen dem tatsächlichen Bedarf privater Kleinvermieter und dem bestehenden Softwareangebot motiviert das vorliegende Projekt. Im Rahmen des Projekts entsteht die Desktop-Anwendung MietManager, die Immobilien, Mieteinheiten, Mieter und Mietverträge verwaltet und als Kernprozess automatisiert eine Nebenkostenabrechnung erstellt.

Daraus ergibt sich die für dieses Projekt leitende Projektfrage. „Inwiefern lässt sich die bislang manuelle, fehleranfällige Nebenkostenabrechnung privater Kleinvermieter durch eine automatisierte Desktop-Anwendung effizienter und fehlerfreier gestalten als mit Tabellenkalkulation oder Papier, und welche Anforderungen ergeben sich daraus für Datenmodell, Architektur und Benutzerschnittstelle?“

1.2 Zielgruppe und Kundennutzen

Zielgruppe sind private Vermieter mit dem in Abschnitt 1.1 beschriebenen Bedarf. Gegenüber einer Tabellenkalkulation liegt der zentrale Mehrwert darin, dass MietManager die Nebenkostenabrechnung nach frei wählbarem Umlageschlüssel, etwa Fläche, Personen oder Anzahl Einheiten, je Kostenart und Mietvertrag automatisch berechnet und dadurch die in der Problemstellung beschriebenen Rechenfehler sowie den Zeitaufwand manueller Berechnung vermeidet. Die Anwendung exportiert das Ergebnis direkt als versandfertigen, personalisierten Geschäftsbrief mit Briefkopf, persönlicher Anrede, Kostenaufschlüsselung, Saldo und Grußformel. Dazu hinterlegt der Vermieter seine eigenen

Stammdaten einmalig in einem Profil, das die Anwendung automatisch als Absender verwendet. Die erstellte Nebenkostenabrechnung kann er anschließend manuell per E-Mail versenden oder ausdrucken und postalisch verschicken.

1.3 Funktionsumfang

Im Kern verwaltet die Anwendung Immobilien sowie Mieter und Mietverträge und prüft dabei automatisch, ob sich Mietverträge derselben Mieteinheit zeitlich überschneiden. Darauf aufbauend erfasst sie die Kostenpositionen einer Immobilie je Abrechnungszeitraum und berechnet daraus die Nebenkostenabrechnung, die sich als versandfertiger PDF-Brief exportieren lässt. Das dafür benötigte Vermieter-Profil lässt sich als Briefkopf-Absender konfigurieren. Zusätzlich zeigt ein Dashboard Kennzahlen zu Immobilien, Einheiten, Leerstand, aktiven Verträgen und Mieteinnahmen. Bewusst nicht Teil dieses Funktionsumfangs sind angrenzende Problemfelder wie Zahlungs- und Mahnwesen, Mehrbenutzerbetrieb und eine mobile Anwendung. Diese Abgrenzung ergibt sich aus der in Abschnitt 1.1 beschriebenen Zielgruppe privater Kleinvermieter mit wenigen Wohneinheiten, für die eine Einzelplatz-Desktopanwendung ausreicht, und wird in der MoSCoW-Priorisierung in Abschnitt 4 im Detail begründet.

1.4 Angestrebtes Ergebnis

Das Projekt strebt eine funktionsfähige Desktop-Anwendung an, die alle in der Anforderungserhebung definierten funktionalen und nichtfunktionalen Anforderungen erfüllt, mit realistischen Testdaten vorbefüllt ist und deren Kernprozess, die automatisierte Nebenkostenabrechnung, sich direkt nach dem Start nachvollziehen lässt.

2 Theoretische Grundlagen

Dieser Abschnitt führt die dem Projektvorgehen zugrunde liegenden Konzepte theoretisch ein, bevor die Abschnitte Risikomanagement, MoSCoW-Priorisierung und Zeitplanung sie konkret auf MietManager anwenden.

2.1 Risikomanagement in Softwareprojekten

Risikomanagement in Softwareprojekten zielt darauf ab, potenzielle Probleme frühzeitig zu erkennen und aktiv zu steuern, statt erst auf ihr Eintreten zu reagieren. Boehm schlägt dazu vor, jedes Risiko anhand zweier Achsen zu bewerten. Diese sind die Eintrittswahrscheinlichkeit und die Schwere der

Auswirkung bei Eintritt (Boehm, 1991, S. 37). Diese Zwei-Achsen-Bewertung unterscheidet sich von einer bloßen, unstrukturierten Liste möglicher Probleme dadurch, dass sie eine begründete Priorisierung der Gegenmaßnahmen erlaubt. Risiken mit hoher Wahrscheinlichkeit und hoher Auswirkung erfordern vorrangig eine Gegenmaßnahme, während gering ausgeprägte Risiken auch unbehandelt tragbar sein können. Abschnitt 3 wendet dieses Bewertungsschema auf die Risiken des vorliegenden Projekts an.

2.2 Anforderungspriorisierung nach MoSCoW

Die MoSCoW-Methode ist ein Verfahren aus dem Anforderungsmanagement, das Anforderungen in die vier Kategorien Muss (Must have), Soll (Should have), Kann (Could have) und Verzicht (Won't have) einteilt (Ahmad et al., 2017, S. 433). Im Unterschied zu einer einfachen numerischen Priorisierung trifft jede Kategorie eine verbindliche Aussage über den Lieferumfang. Muss-Anforderungen sind für ein erfolgreiches Projektergebnis zwingend erforderlich, während Kann-Anforderungen bei Zeitdruck ohne Gefährdung des Projekterfolgs entfallen können. Gerade unter der in Abschnitt 3 identifizierten Zeitknappheit erlaubt diese Abstufung, den Funktionsumfang im Projektverlauf kontrolliert zu reduzieren, statt unter Zeitdruck an allen Anforderungen gleichermaßen Abstriche machen zu müssen. Abschnitt 4 wendet die Methode auf den Funktionsumfang von MietManager an.

2.3 Iteratives und inkrementelles Vorgehen

Iterative und inkrementelle Softwareentwicklung liefert den Projektumfang nicht in einem einzigen, vorab vollständig geplanten Durchlauf, sondern in aufeinanderfolgenden Zwischenergebnissen, die jeweils überprüft und bei Bedarf angepasst werden, bevor der nächste Abschnitt beginnt. Im Unterschied zu einem Wasserfallmodell, bei dem Fehler in frühen Phasen oft erst spät im Projekt sichtbar werden, lassen sich Rückmeldungen zu einem Zwischenergebnis so bereits vor Abschluss des Gesamtprojekts berücksichtigen. Abschnitt 5 beschreibt, wie dieses Vorgehen über drei Portfolio-Phasen mit jeweils eingeholtem Tutor-Feedback konkret umgesetzt wird.

3 Risikomanagement

Tabelle 3.1 wendet das in Abschnitt 2.1 eingeführte Bewertungsschema aus Eintrittswahrscheinlichkeit und Auswirkung auf die Risiken des vorliegenden Projekts an. Die beiden am höchsten eingestuften Risiken, R2 (Wartezeit auf Tutor-Feedback) und R5 (Zeitknappheit durch Studium und Beruf), sind beide unmittelbare Folgen der knappen Gesamtdauer von vier Wochen, und werden daher durch dieselbe übergeordnete Gegenmaßnahme adressiert. Die Portfolioteile werden jeweils so früh

wie möglich eingereicht, was sowohl Pufferzeit für Feedback schafft als auch die in Abschnitt 2.2 eingeführte MoSCoW-Priorisierung nutzbar macht, um den Funktionsumfang bei Bedarf kontrolliert zu reduzieren. Für keines der sechs identifizierten Risiken wird auf eine Gegenmaßnahme verzichtet, da laut dem in Abschnitt 2.1 eingeführten Bewertungsschema erst die Kombination aus niedriger Wahrscheinlichkeit und niedriger Auswirkung ein Tolerieren ohne Gegenmaßnahme rechtfertigen würde. Selbst R6 (Datenverlust) erfordert trotz niedriger Eintrittswahrscheinlichkeit wegen seiner hohen Auswirkung eine Gegenmaßnahme.

#	Risiko	Wahrsch.	Auswirkung	Gegenmaßnahme
R1	Zu viele Features geplant	mittel	hoch	MoSCoW-Priorisierung
R2	Wartezeit auf Tutor-Feedback verzögert Zeitplan	hoch	hoch	Portfolioteile 1 und 2 möglichst früh statt am letzten Tag einreichen, um innerhalb der Woche noch auf Feedback reagieren zu können
R3	Berechnungslogik der Nebenkostenabrechnung unterschätzt	mittel	hoch	Rechenprototyp früh in Woche 3 bauen, bevor die GUI angebunden wird
R4	Packaging-Probleme	mittel	mittel	PyInstaller bereits in Woche 3 testen, nicht erst am Ende
R5	Zeitknappheit durch Studium/Beruf bei nur 4 Wochen Gesamtdauer	hoch	hoch	Klare Priorisierung nach MoSCoW, keine Pufferzeit für Nice-to-have-Features einplanen
R6	Datenverlust	niedrig	hoch	Bereits zu Beginn der Implementierung GitHub mit regelmäßigen, kleinteiligen Commits nutzen

Tabelle 3.1: Risikomanagement

4 MoSCoW-Priorisierung

Abschnitt 2.2 führt die MoSCoW-Methode als Reaktion auf das in Abschnitt 3 identifizierte Risiko von zu vielen geplanten Features ein. Tabelle 4.1 wendet sie auf den Funktionsumfang von MietManager an. Die Muss-Anforderungen entsprechen dabei genau den User Stories FA-01 bis FA-07 aus dem Anforderungsdokument, da ohne sie der in der Problemstellung beschriebene Kernnutzen, die automatisierte Nebenkostenabrechnung, nicht realisiert wäre. Das Dashboard (FA-08) wurde dagegen als Soll statt als Muss eingestuft, weil es den bereits vollständigen Kernprozess lediglich um eine zusätzliche Übersicht ergänzt, ohne selbst Voraussetzung für die Abrechnungserstellung zu sein. E-Mail-Versand und Kennzahlen-Export wurden als Kann eingestuft und bewusst aus dem Muss-Umfang ausgeschlossen, da beide den bestehenden Kernprozess nur bequemer, aber nicht grundsätzlich anders nutzbar machen.

Kategorie	Umfang
Muss (Must have)	Immobilien- und Mieteinheitenverwaltung, Mieterverwaltung, Mietvertragsverwaltung mit Überlappungsprüfung, Kostenpositionserfassung, automatische Berechnung der Nebenkostenabrechnung, PDF-Export als versandfertiger Geschäftsbrief, Vermieterprofil als Briefkopf-Absender (FA-01 bis FA-07).
Soll (Should have)	Dashboard mit Übersicht zu Immobilien, Einheiten, Leerstand, aktiven Verträgen sowie Mieteinnahmen (FA-08).
Kann (Could have)	Automatischer E-Mail-Versand der Abrechnung direkt aus der Anwendung heraus, CSV-/Excel-Export der Kennzahlen.
Verzicht (Won't have)	Zahlungs- und Mahnwesen, Mehrbenutzerbetrieb, mobile Anwendung.

Tabelle 4.1: MoSCoW-Priorisierung des Funktionsumfangs

5 Zeitplanung

Abschnitt 2.3 führt das iterative und inkrementelle Vorgehen ein, dem das Projekt folgt. Die drei Portfolio-Phasen liefern jeweils ein abgeschlossenes Zwischenergebnis, zu dem vor der nächsten Phase Tutor-Feedback eingeholt und eingearbeitet wird. Das Projekt ist auf eine Gesamtdauer von vier Wochen angesetzt (Mo. 27.07.2026 bis So. 23.08.2026). Die Konzeptionsphase erhält davon zwei Kalenderwochen, da hier mit Projektdokumentation, Anforderungsdokument und Spezifikationsdokument die meisten Dokumente entstehen und eine sorgfältige Planung für den weiteren Projektverlauf entscheidend ist. Die Erarbeitungs- und Finalisierungsphase erhalten je eine Kalenderwoche. Trotz der zusätzlichen Woche bleibt der Puffer insgesamt knapp (siehe Risiken R2 und R5), weshalb die einzelnen Portfolioteile jeweils so früh wie möglich fertiggestellt werden, um noch Zeit für Tutor-Feedback und Korrekturen zu haben. Abbildung 5.1 zeigt den Projektzeitplan als Gantt-Diagramm. Anschließend werden die vier Wochen inhaltlich beschrieben. Der Projektfortschritt lässt sich an den drei Meilensteinen der Portfolioabgabe ablesen. Der Einreichung von Portfolioteil 1 (Konzeptionsphase) am 09.08.2026, von Portfolioteil 2 (Erarbeitungs-/Reflexionsphase) am 16.08.2026 und von Portfolioteil 3 (Finalisierungsphase) am 23.08.2026.

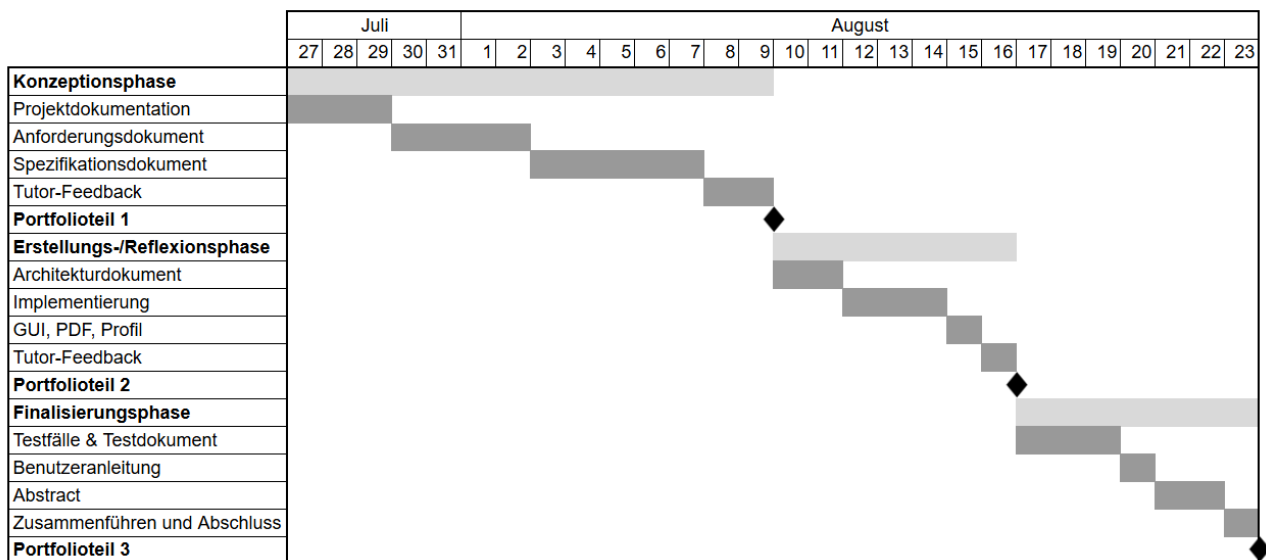


Abbildung 5.1: Projektzeitplan

5.1 Woche 1 – Konzeptionsphase, Teil 1 (27.07.–02.08.2026)

In der ersten Woche werden zunächst Projektübersicht, Risikomanagement, MoSCoW-Priorisierung und Zeitplanung als Grundlage der Projektdokumentation erstellt. Anschließend werden für das Anforderungsdokument die Stakeholder identifiziert, User Stories gesammelt, die nichtfunktionalen Anforderungen definiert und die funktionalen Anforderungen als UML-Use-Case-Diagramm visualisiert.

5.2 Woche 2 – Konzeptionsphase, Teil 2 (03.08.–09.08.2026)

In der zweiten Woche entsteht das Spezifikationsdokument. Das Datenmodell wird erarbeitet und als UML-Klassendiagramm visualisiert, der Kernprozess „Nebenkostenabrechnung erstellen“ als UML-Aktivitätsdiagramm dargestellt sowie Geschäftsregeln, Systemschnittstellen und Benutzerschnittstellen inklusive GUI-Mockups beschrieben. Vor Abgabe wird Tutor-Feedback zu allen drei Dokumenten der Konzeptionsphase eingeholt. Das Ergebnis der Konzeptionsphase ist die Einreichung von Portfolioteil 1.

5.3 Woche 3 – Erarbeitungs-/Reflexionsphase (10.08.–16.08.2026)

In Woche 3 wird das Architekturdokument erstellt und die Struktur des GitHub-Repositorys aufgesetzt. Im Zentrum der Woche steht anschließend die Implementierung. Datenmodell und Datenbank, CRUD-Funktionen, die Berechnungslogik der Nebenkostenabrechnung, die grafische Oberfläche, der PDF-Export sowie das Vermieter-Profil werden umgesetzt. Vor der Abgabe wird Tutor-Feedback eingeholt. Die Erarbeitungs- und Reflexionsphase endet mit der Einreichung von Portfolioteil 2.

5.4 Woche 4 – Finalisierungsphase (17.08.–23.08.2026)

Die vierte Woche beginnt mit der Einarbeitung des Tutor-Feedbacks aus Phase 2. Anschließend werden Testfälle geschrieben und im Testdokument protokolliert sowie die Benutzeranleitung verfasst. Zum Abschluss entsteht das zweiseitige Abstract, bevor alle Dokumente zusammengeführt und eine ZIP-Datei aus dem Inhalt des GitHub-Repositorys erzeugt werden. Die Finalisierungsphase signalisiert den Abschluss des Projekts und endet mit der Einreichung von Portfolioteil 3.

6 GitHub-Repository

Der vollständige Quellcode, die Tests sowie alle zur Ausführung der Anwendung notwendigen Dateien sind im folgenden GitHub-Repository abgelegt: https://github.com/Michael5601/PSE_Abgabe_MietManager

7 Benutzeranleitung

Der folgende Abschnitt beschreibt zunächst Installation und Start von MietManager, bevor die Bedienung der wichtigsten Tabs anhand des typischen Arbeitsablaufs erläutert wird.

7.1 Installation und Start

Für die Nutzung von MietManager auf einem Windows-10/11-Rechner gibt es zwei Wege.

Am einfachsten ist der Start über die im GitHub-Repository unter `dist/MietManager.exe` bereitgestellte eigenständige Anwendung. Sie wurde mit PyInstaller aus dem Quellcode gebaut und bündelt bereits sämtliche Laufzeitabhängigkeiten. Auf dem Zielrechner müssen daher weder Python noch zusätzliche Bibliotheken installiert werden, ein Doppelklick auf die Datei startet die Anwendung direkt.

Alternativ lässt sich MietManager aus dem Quellcode ausführen. Dafür wird Python 3.14 oder neuer benötigt. Sind auf dem Rechner mehrere Python-Versionen installiert, sollte die virtuelle Umgebung über den Windows-Python-Launcher mit expliziter Versionsangabe angelegt werden, da der einfache Befehl `python` je nach PATH-Reihenfolge auch eine ältere Version auflösen kann. Folgende Befehle klonen das Repository, wechseln in den dabei neu angelegten Projektordner und richten dort eine virtuelle Umgebung mit allen Abhängigkeiten ein.

- `git clone https://github.com/Michael5601/PSE_Abgabe_MietManager.git`
- `cd PSE_Abgabe_MietManager`

- `py -3.14 -m venv .venv`
- `.venv\Scripts\activate`
- `pip install -e ".[dev]"`

Der Befehl `mietmanager` startet anschließend die Anwendung. Dieser Weg eignet sich vor allem, um den Quellcode nachzuvollziehen, die automatisierten Tests mit `pytest` auszuführen oder mit `python scripts/performance_benchmark.py` den im Testdokument beschriebenen Performance-Benchmark gegen eine an der dokumentierten Skalierungsgrenze geseedete Datenbank zu wiederholen.

Beim ersten Start legt MietManager unabhängig vom gewählten Weg automatisch eine lokale SQLite-Datenbank unter `%USERPROFILE%\mietmanager\mietmanager.db` an und befüllt sie mit realistischen Beispieldaten zu Immobilien, Mietern, Mietverträgen und Kostenpositionen. Dadurch lässt sich der in Abschnitt 1.1 beschriebene Kernprozess, das Erstellen einer Nebenkostenabrechnung, direkt nach dem Start nachvollziehen, ohne zunächst manuell Daten anlegen zu müssen. Bei jedem weiteren Start greift die Anwendung auf dieselbe Datenbank zu, sodass alle Eingaben dauerhaft erhalten bleiben.

7.2 Bedienung

Die Anwendung gliedert sich in sechs Tabs, die in der Reihenfolge des typischen Arbeitsablaufs angeordnet sind: Dashboard, Immobilien, Mieter, Mietverträge, Kostenpositionen und Nebenkostenabrechnung. Vor der ersten Nutzung sollte über den Profil-Button oben rechts das Vermieterprofil mit Name, Straße, PLZ und Ort ausgefüllt werden, da diese Angaben als Absender jeder exportierten Nebenkostenabrechnung dienen. Im Tab Immobilien legt „Neue Immobilie“ zunächst das Gebäude mit Bezeichnung und Adresse an. Durch Auswahl der Immobilie und „Neue Mieteinheit“ lassen sich anschließend die einzelnen Wohneinheiten mit Fläche und Zimmerzahl ergänzen. Im Tab Mieter werden die Kontaktdaten der Mietparteien erfasst. Im Tab Mietverträge verknüpft ein neuer Mietvertrag eine Mieteinheit mit einem Mieter und legt Beginn, Kaltmiete, Nebenkosten-Vorauszahlung, Kautions- und Personenzahl fest. Überschneidet sich der gewählte Zeitraum mit einem bestehenden Mietvertrag derselben Mieteinheit, lehnt die Anwendung das Speichern automatisch ab. Im Tab Kostenpositionen werden die anfallenden Betriebskosten je Immobilie, Zeitraum und Umlageschlüssel erfasst, etwa Grundsteuer nach Fläche oder Müllabfuhr nach Personenzahl. Liegen für eine Immobilie sowohl Kostenpositionen als auch aktive Mietverträge im gewünschten Abrechnungszeitraum vor, berechnet der Tab Nebenkostenabrechnung nach Auswahl von Immobilie und Zeitraum sowie einem Klick auf „Abrechnung erstellen“ die Kostenverteilung. Die Ergebnistabelle zeigt je Mietvertrag den Kostenanteil, die geleistete Vorauszahlung und den daraus resultierenden Saldo. Über „Als PDF exportieren...“

lässt sich die Abrechnung anschließend als versandfertiger, personalisierter Brief je Mietvertrag in einem frei wählbaren Zielordner speichern. Bereits erstellte Abrechnungen bleiben im Verlauf desselben Tabs einsehbar und lassen sich dort bei Bedarf auch wieder löschen. Das Dashboard fasst währenddessen jederzeit die wichtigsten Kennzahlen zu Immobilien, Einheiten, Leerstand, aktiven Verträgen und Mieteinnahmen zusammen.

8 Fazit

Dieses Dokument leitet ausgehend vom Managementproblem privater Kleinvermieter die Projektfrage, Zielgruppe und den Kundennutzen von MietManager her und grenzt den Funktionsumfang mittels MoSCoW-Priorisierung ab. Darauf aufbauend beschreibt das Anforderungsdokument Stakeholder sowie funktionale und nichtfunktionale Anforderungen, das Spezifikationsdokument konkretisiert Datenmodell und Geschäftsprozesse, das Architekturdokument begründet die gewählte Drei-Schichten-Architektur gegenüber den betrachteten Alternativen und das Testdokument weist die Erfüllung der Anforderungen anhand von 42 automatisierten und 28 manuellen Testfällen nach.

Die eingangs in Abschnitt 1.1 gestellte Projektfrage lässt sich auf dieser Grundlage positiv beantworten. Anstelle einer manuellen Verteilung der Kostenpositionen nach Umlageschlüssel und eines manuellen Abgleichs mit den geleisteten Vorauszahlungen, beides in der Problemstellung als fehleranfällige und zeitaufwendige Schritte beschrieben, übernimmt die implementierte Anwendung Berechnung, Abgleich und den Export als versandfertigen, personalisierten PDF-Geschäftsbrief vollständig automatisiert. Alle sieben als Muss eingestuften funktionalen Anforderungen sind ebenso umgesetzt wie die als Soll eingestufte Dashboard-Anforderung, beides ist durch das Testdokument mit 42 automatisierten und 28 manuellen Testfällen verifiziert, wodurch die Fehlerfreiheit der Berechnung reproduzierbar nachgewiesen ist. Welche Anforderungen sich daraus für Datenmodell, Architektur und Benutzerschnittstelle ergeben, beschreiben Anforderungs-, Spezifikations- und Architekturdokument im Detail, wie eingangs zusammengefasst. Eine Desktop-Anwendung kann private Kleinvermieter damit im beschriebenen Umfang tatsächlich dabei unterstützen, die bislang manuelle Nebenkostenabrechnung effizienter und fehlerfreier zu gestalten als mit Tabellenkalkulation oder Papier.

Aus den einzelnen Dokumenten lassen sich zwei Arten von Limitationen unterscheiden. Zum einen bestehen bewusst getroffene Scope-Entscheidungen, die keine technische Schuld darstellen, sondern die in der MoSCoW-Priorisierung begründete Abgrenzung auf die Zielgruppe privater Kleinvermieter widerspiegeln. Dazu zählen der Verzicht auf Zahlungs- und Mahnwesen, auf Mehrbenutzerbetrieb und auf eine mobile Anwendung. Zum anderen bestehen technische Grenzen, die aus den getroffenen Architekturentscheidungen resultieren. Die dateibasierte SQLite-Datenbank legt die in den nichtfunktionalen Anforderungen quantifizierte Skalierungsgrenze von etwa 20 Mieteinheiten und 200

Mietverträgen fest und schließt gleichzeitigen Mehrbenutzerzugriff aus. Eine Datensicherung ist bislang nur manuell durch Kopieren der Datenbankdatei möglich, eine automatisierte Backup-Funktion fehlt. Auch die im Testdokument begründete Entscheidung, die UI- und Datenzugriffsschicht nicht automatisiert, sondern nur durch manuelle Systemtests abzusichern, stellt eine bewusst eingegangene Grenze der Qualitätssicherung dar. Zwei der als Kann eingestuften Funktionen, der automatische E-Mail-Versand und der Kennzahlen-Export, wurden im Rahmen dieses Projekts nicht umgesetzt.

Aus dem Projektverlauf lassen sich mehrere Erkenntnisse für künftige Softwareprojekte ableiten. Die im Architekturdokument begründete Entscheidung für eine strikte Drei-Schichten-Architektur bestätigte sich als tragfähig, weil sie die Geschäftslogik unabhängig von der Benutzeroberfläche testbar machte, noch bevor die grafische Oberfläche überhaupt existierte. Umgekehrt zeigte sich, dass eine Zeitplanung, die vor allem die eigene Bearbeitungszeit kalkuliert, den tatsächlichen Kalenderaufwand systematisch unterschätzt, sobald die Bearbeitung wie in diesem Projekt nur in begrenzten Zeitfenstern neben Studium und Beruf möglich ist. Schließlich bestätigte sich, dass Usability-Schwächen wie die ursprünglich manuellen statt automatischen Aktualisierungen in der Oberfläche oft erst durch die tatsächliche Nutzung der fertigen Anwendung sichtbar werden und sich nicht vollständig durch Mock-ups in der Konzeptionsphase vorwegnehmen lassen.

Für eine Weiterentwicklung über den Rahmen dieses Portfolios hinaus ergeben sich daraus konkrete Handlungsempfehlungen. Kurzfristig ließen sich die bereits als Kann eingestuften Funktionen, der automatische E-Mail-Versand und der CSV-Export der Dashboard-Kennzahlen, ohne Änderung der bestehenden Architektur ergänzen. Mittelfristig sollte eine automatisierte Backup-Funktion die bislang manuelle Datensicherung ablösen, um dem in den nichtfunktionalen Anforderungen beschriebenen Risiko eines Datenverlusts weiter vorzubeugen. Sollte der adressierte Kundenstamm über die dokumentierte Skalierungsgrenze hinauswachsen oder Mehrbenutzerbetrieb erforderlich werden, müsste die dateibasierte SQLite-Anbindung durch eine Client-Server-Datenbank ersetzt werden, wofür die bereits bestehende Trennung von Service- und Datenzugriffsschicht eine geeignete Grundlage bietet. Auch die Testabdeckung ließe sich erweitern, etwa durch den Einsatz von Werkzeugen wie pytest-qt, um künftig auch die bislang nur manuell geprüfte Benutzeroberfläche automatisiert abzusichern.

Literaturverzeichnis

- Ahmad, K. S., Ahmad, N., Tahir, H., & Khan, S. (2017). Fuzzy_MoSCoW: A fuzzy based MoSCoW method for the prioritization of software requirements. *International Conference on Intelligent Computing, Instrumentation and Control Technologies (ICICICT)*, 433–437. <https://doi.org/10.1109/ICICICT1.2017.8342602>
- Boehm, B. W. (1991). Software risk management: principles and practices. *IEEE Software*, 8(1), 32–41. <https://doi.org/10.1109/52.62930>